

Παράλληλος και Κατανεμημένος Υπολογισμός I

Μοιραζόμενη μνήμη Αμοιβαίος αποκλεισμός (συνέχεια)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Java : Αμοιβαίος αποκλεισμός με `synchronized`

Αμοιβαίος αποκλεισμός με synchronized

- Ένα synchronized τμήμα, καθορίζει και αυτό αμοιβαίο αποκλεισμό όπως ένα ζεύγος lock - unlock:

```
Lock lock =  
new ReentrantLock();
```

```
Object anObject =  
new(Object);
```

```
lock.lock();  
try {  
  
    //critical section  
  
} finally {  
    lock.unlock();  
}
```

```
...  
synchronized(anObject) {  
  
    //critical section  
  
}
```

- Όμως το synchronized δεν κλειδώνει ένα κλείδωμα, δηλαδή ένα ReentrantLock αλλά ένα αντικείμενο.

Εγγενές κλείδωμα αντικειμένου 1

- Οποιοδήποτε αντικείμενο στη Java διαθέτει ένα **εγγενές κλείδωμα** (intrinsic lock) που κληρονομεί από την κλάση Object.
- Το κλείδωμα αυτό υπάρχει αλλά δε χρησιμοποιείται όταν δε χρησιμοποιούμε τη λέξη synchronized.
- Επομένως η χρήση του synchronized σημαίνει χρήση του εγγενούς κλειδώματος που διαθέτει κάθε αντικείμενο της Java.
- Γενικά το synchronized πάντα αναφέρεται σε κάποιο αντικείμενο.

Synchronized block 1

- Επομένως, στη δήλωση οποιασδήποτε κλάσης μπορούμε στο κώδικα μιας μεθόδου να έχουμε ένα `synchronized block`:

```
class aClass {  
    ...  
    public void aMethod(){  
        ...  
        synchronized (this) {  
            ...  
        }  
    }  
}
```

- Κάθε αντικείμενο αυτής της κλάσης όταν δημιουργείται διαθέτει ένα εγγενές κλείδωμα
`aClass anObject = new aClass();`

Synchronized block 2

- Έστω ότι κάποιο νήμα εκτελεί τη κλήση:
... `anObject.aMethod()`;
- Κατά την εκτέλεση της μεθόδου, όταν το νήμα φθάσει στη λέξη `synchronized` για να συνεχίσει πρέπει να καταλάβει το εγγενές κλείδωμα του αντικειμένου `anObject`.
- Το `this` αναφέρεται στο αντικείμενο του οποίου η μέθοδος έχει κληθεί.
- Αν κάποιο άλλο νήμα κατέχει το κλείδωμα τότε το νήμα θα ανασταλεί σε αναμονή αποδέσμευσης του κλειδώματος ακριβώς όπως με το `ReentrantLock`.

Synchronized method 1

- Η λέξη `synchronized` μπορεί να χρησιμοποιηθεί όχι μόνο σε τμήματα κώδικα αλλά και σε δηλώσεις μεθόδων, πχ

```
public synchronized void aMethod(){  
    ...  
}
```

- Σε αυτή τη περίπτωση το νήμα που καλεί τη μέθοδο `aMethod` πρέπει να καταλάβει το εγγενές κλείδωμα του αντικειμένου στο οποίο αναφέρεται.
- Επομένως έχουμε `synchronized blocks` και `synchronized methods`.

Αμοιβαίος αποκλεισμός με `synchronized` 2

Συμπερασματικά:

- Όταν ένα νήμα εκτελεί `synchronized` κώδικα τότε καταλαμβάνει το εγγενές κλείδωμα του αντικειμένου αναφοράς. Έτσι κανένα άλλο νήμα δεν μπορεί να εκτελέσει `synchronized` κώδικα στο συγκεκριμένο αντικείμενο.
- Το εγγενές κλείδωμα αναφέρεται στη σχέση νήμα – αντικείμενο, όχι στη σχέση νήμα – μέθοδος (ή τμήμα κώδικα). Αν υπάρχουν πολλές `synchronized` μέθοδοι (ή τμήματα) που σχετίζονται με αντικείμενο αναφοράς τότε όλα ελέγχονται από το συγκεκριμένο νήμα που έχει καταλάβει το κλείδωμα.



Παραδείγματα αμοιβαίου αποκλεισμού με synchronized

- Code/SharedCounterSync

Διακριτότητα synchronized 1

- Το `synchronized` προσφέρει το πλεονέκτημα της κοινής αντιμετώπισης όλων των μεταβλητών που σχετίζονται με ένα αντικείμενο. Πχ.

```
Class Stack {  
    int top;  
    int [] elements;  
  
    ...  
    synchronized void push(int x) {  
        top++; elements[top] = x;  
    }  
    synchronized int pop () {  
        int x = element[top]; top--; return x;  
    }  
    ...  
}
```

Διακριτότητα synchronized 2

- Όμως αν απαιτούνται ταυτόχρονες ενέργειες σε πολλά σημεία μιας δομής δεδομένων του αντικειμένου, το `synchronized` δεν είναι εύκολο να τις χειριστεί: κλειδώνει ολόκληρο το αντικείμενο. Πχ.

```
Class Queue {  
    int front, back;  
    int [] elements;  
  
    ...  
    synchronized void put(int x) {  
        back=(back++)%size; elements[back] = x;  
    }  
    synchronized int get () {  
        int x = element[front];  
        front=(front++)%size; return x;  
    }  
    ...  
}
```

Διακριτότητα synchronized 3

- Μια λύση: χρήση lock Objects

```
Class Queue {  
    int front, back;  
    Object front_l = new Object();  
    Object back_l = new Object();  
    int [] elements;  
    ...  
    void put(int x) {  
        synchronized (back_l) {  
            back=(back++)%size; elements[back] = x;  
        }  
    }  
    int get () {  
        synchronized (front_l) {  
            int x = element[front];  
            front=(front++)%size; return x;  
        }  
    }  
}
```

Μη-συμβατικές εφαρμογές του synchronized 1

- Στη Java τα νήματα είναι αντικείμενα. Επομένως έχουν και αυτά εγγενές κλείδωμα.
- Αν μια μέθοδος που εκτελούν τα νήματα -μέσω της run()- είναι static, τότε όλα τα νήματα εκτελούν τον ίδιο κώδικα.
- Αν μια static μέθοδος δηλωθεί synchronized το νήμα που την εκτελεί αποτρέπει τα άλλα νήματα να την εκτελέσουν.

Μη-συμβατικές εφαρμογές του `synchronized` 2

- Μερικές φορές καταφεύγουμε στη χρήση του `synchronized(this.getClass())` κατάληψη του κλειδώματος της κλάσης. Αν αυτό είναι απαραίτητο καλύτερα να γράφουμε `synchronized(MyThread.class)` για να φαίνεται καθαρά ποια κλάση κλειδώνουμε.
- Οι τεχνικές αυτές χρησιμοποιούνται όταν δεν είναι εύκολο να βρεθεί αντικείμενο αναφοράς για το `synchronized` (πχ όταν κλειδώνουμε εκτυπώσεις!)



Παραδείγματα μη συμβατικών εφαρμογών αμοιβαίου αποκλεισμού με `synchronized`

- `Code/SharedCounterSyncAlt`

Παρατηρήσεις synchronized 1

- Το `synchronized` προσφέρει αντικειμενοστρεφή αμοιβαίο αποκλεισμό και είναι συμβατό με τη λογική του επόπτη (`monitor`) ως βασικού μηχανισμού ταυτοχρονισμού στη Java. (Θα δούμε αναλυτικά τους επόπτες αργότερα).
- Δεν απαιτεί εμφανή δήλωση και χρήση κλειδώματος.
- Επιτρέπει εύκολο αμοιβαίο αποκλεισμό με την απλή πρόσθεση μιας λέξης στις μεθόδους της κλάσης του προστατευόμενου αντικειμένου.
- Διαχειρίζεται αυτόματα την απελευθέρωση κλειδώματος από εξαιρέσεις, διακοπές, `break`, `continue`, `return`.

Παρατηρήσεις synchronized 2

Όμως έχει δεχτεί κριτική για:

- τη μικρή διακριτότητα σε σύνθετες δομές δεδομένων
- την ανάγκη να έχουμε Lock-Unlock στην ίδια μέθοδο
- την έλλειψη ελέγχου (tryLock) και timeout
- την έλλειψη δικαιοσύνης ..

Στη βιβλιοθήκη `java.util.concurrent` προσφέρονται πολλά επιπλέον κλειδώματα με περισσότερες λειτουργίες. Πχ, το `ReentrantLock` έχει αυτές τις επιπλέον δυνατότητες που δεν περιγράψαμε εδώ.

Τυπικοί σκελετοί προγραμμάτων με αμοιβαίο αποκλεισμό

Συνδυασμοί των παρακάτω

- Μοιραζόμενες μεταβλητές:

Καθολικές μεταβλητές, Πολλαπλά ορίσματα στο main, Μοιραζόμενη δομή, Μοιραζόμενο αντικείμενο

- Αμοιβαίος αποκλεισμός:

Lock, Mutex, Synchronized με lockObject, Synchronized



Αμοιβαίος αποκλεισμός: μερικές εφαρμογές

Κλειδώματα και πολλαπλές κρίσιμες περιοχές 1

- **Συνεπές κλείδωμα:** Ένα κλείδωμα προστατεύει πλήρως συγκεκριμένες θέσεις μνήμης, κατά την εκτέλεση όλου του κώδικα.
- Κάθε κλείδωμα αντιστοιχεί σε μια μοιραζόμενη μεταβλητή (ή σύνολο μεταβλητών) και την παρακολουθεί σε όλες τις τροποποιήσεις της.
- Όταν ένα νήμα καταλάβει ένα κλείδωμα όλα τα άλλα νήματα δεν πρέπει να μπορούν να τροποποιήσουν τη μεταβλητή, ανεξάρτητα από το αν εκτελούν την ίδια ή άλλη κρίσιμη περιοχή, που αναφέρεται στην ίδια μεταβλητή.
- Οι ίδιες παρατηρήσεις ισχύουν και για τον αμοιβαίο αποκλεισμό με δυαδικό σηματοφόρο (mutex).

Κλειδώματα και πολλαπλές κρίσιμες περιοχές 2

```
class bankAccount {  
    int balance = ...;  
    lock_type l;  
  
    //constructor  
    ...  
  
    int get_balance() {  
        Lock(l);                }  
        return balance;  
        Unlock(l);  
    }  
  
    deposit (int amount) {  
        Lock(l);  
        balance += amount;  
        Unlock(l);  
    }  
  
    withdraw (int amount) {  
        Lock(l);  
        if (get_balance() >= amount)  
            balance -= amount;  
        Unlock(l);  
    }  
}
```

Κλειδώματα και πολλαπλές κρίσιμες περιοχές 3

// shared bankAccount S

Νήμα A

Νήμα B

Νήμα C

...

...

...

S.deposit(x);

S.withdraw(y);

z = S.get_balance();

....

...

...

Μια μοιραζόμενη δομή μπορεί να προσπελαστεί από τρεις συναρτήσεις. Τρία κρίσιμα τμήματα, κοινό κλείδωμα: προστατεύουμε τα δεδομένα. Μόνο μια συνάρτηση μπορεί να εκτελεστεί κάθε φορά. Γιατί κλειδώνουμε και το get_balance();

Κλείδωμα επανεισόδου

Reentrant Lock

Η κλήση της μεθόδου `get_balance` μέσα από τη κλήση `withdraw` απαιτεί επανακατάληψη του κλειδώματος.

- Ο κάτοχος ενός κλειδώματος επανεισόδου μπορεί να το ξανακαταλάβει χωρίς να το απελευθερώσει ενδιάμεσα.
- Αποφεύγουμε ενδιάμεση κατάληψη από άλλο νήμα.
- Το κλείδωμα επανεισόδου έχει μετρητή που κρατά τον αριθμό των καταλήψεων.
- Τελικά το κλείδωμα επανεισόδου θα πρέπει να απελευθερωθεί όσες φορές έχει καταληφθεί.

Επιστροφές, σφάλματα, εξαιρέσεις

```
class bankAccount {  
    ...  
    deposit (int amount) {  
        Lock(l);  
        if (amount < 0) {  
            Unlock(l);  
            Exception(negativeAmount);  
        }  
        balance += amount;  
        Unlock(l);  
    }  
    withdraw (int amount) {  
        Lock(l);  
        if (get_balance() < amount) {  
            Unlock(l);  
            Exception(InadequateBalance);  
        }  
        balance -= amount;  
        Unlock(l);  
    }  
    ...  
}
```

Προσοχή στη διαχείριση
σφαλμάτων, εξαιρέσεων,
return, break, continue.

Η έξοδος έξοδος από μια
μέθοδο ή έναν βρόχο
πρέπει να απελευθερώνει
το κλείδωμα, δηλαδή να
περιλαμβάνει Unlock.

Ανεπαρκές εύρος κλειδώματος

```
class bankAccount {  
    ...  
    withdraw (int amount) {  
        if (get_balance() < amount) {           // A. lock() - unlock()  
            Exception(negativeAmount);  
        }  
        Lock(l);                                // B. lock() - unlock()  
        balance -= amount;  
        Unlock(l);  
    }  
    ...  
}
```

Προσοχή στα διαδοχικά κλειδώματα μικρού εύρους, όπως τα A, B. Δεν υπάρχει εγγύηση ότι το κλείδωμα A θα συμβεί αμέσως μετά το κλείδωμα B. Μπορεί να παρεμβληθεί η εκτέλεση άλλου κώδικα που να τροποποιήσει το balance, έτσι ώστε το balance -= amount να είναι πια λανθασμένο.

Διακριτότητα (granularity) κλειδωμάτων

Όταν έχουμε πχ έναν πίνακα τι πρέπει να κλειδώσουμε;

- **Κλείδωμα μικρής διακριτότητας.** Όλη τη δομή: εύκολη λύση αλλά μείωση του ταυτοχρονισμού.
- **Κλείδωμα μεγάλης διακριτότητας.** Κάθε στοιχείο της δομής: δυσκολότερη λύση (πρέπει να διαχειριστούμε δομή κλειδωμάτων) αλλά αύξηση του ταυτοχρονισμού.

Κλειδώματα και πίνακες (arrays): Λεπτομερές κλείδωμα

```
// shared bankAccount S [1000]  
// 1000 locks
```

Νήμα A	Νήμα B	Νήμα C
...	...	...
S[2].deposit(x);	S[30].withdraw(y);	z=S[76].get_balance();
....	...	...

Κλείδωμα κάθε γραμμής όλου του πίνακα: Το κλείδωμα βρίσκεται μέσα στο αντικείμενο bankAccount. Πιο σύνθετη λύση, προσφέρει λεπτομερή Παραλληλισμό. Οι τρεις λειτουργίες μπορούν να εκτελεστούν παράλληλα Αφού αναφέρονται σε διαφορετική γραμμή, άρα καταλαμβάνουν διαφορετικό κλείδωμα η κάθε μια.

Κλειδώματα και πίνακες (arrays):

Αδρομερές κλείδωμα 1

```
class altBankAccount {  
  
    int[] balance[1000];  
    lock_type l; // 1 lock  
  
    //constructor  
    ...  
  
    int get_balance (int i) {  
        Lock(l);  
        return balance[i];  
        Unlock(l;)  
    }  
  
    deposit (int amount, int i) {  
        Lock(l);  
        balance[i] += amount;  
        Unlock(l;)  
    }  
  
    withdraw (int amount, int i) {  
        Lock(l);  
        if (get_balance[i] >= amount)  
            balance[i] -= amount;  
        Unlock(l;)  
    }  
}
```

Κλειδώματα και πίνακες (arrays): Αδρομερές κλείδωμα 2

```
// shared altbankAccount S
```

Νήμα A

...

S.deposit(x,2);

....

Νήμα B

...

S.withdraw(y,30);

...

Νήμα C

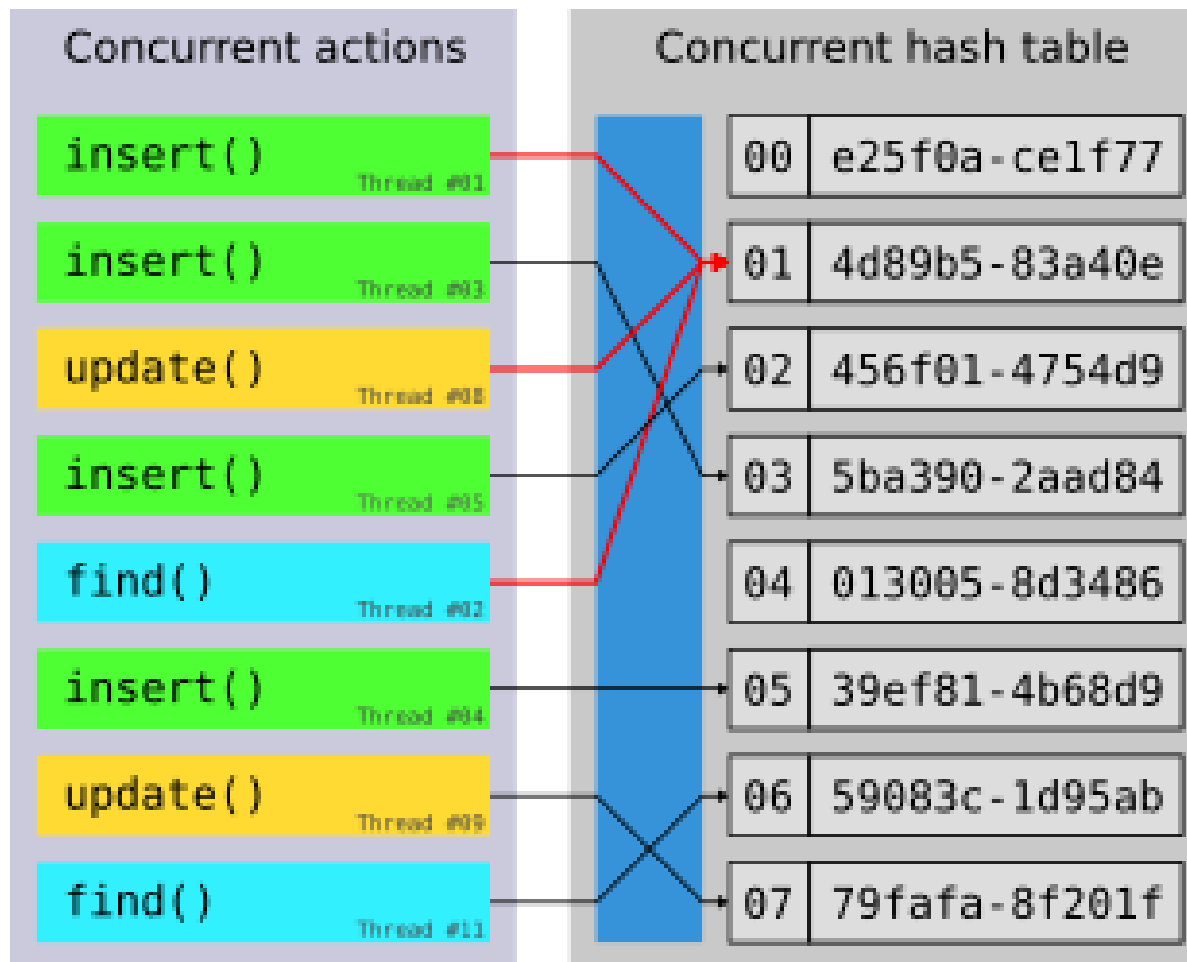
...

z=S.get_balance(76);

...

Κλείδωμα όλου του πίνακα: πιο απλό, αλλά μικρότερος παραλληλισμός. Αφού έχουμε μόνο ένα κλείδωμα, μόνο μια συνάρτηση Μπορεί να εκτελεστεί, ανεξάρτητα από τη γραμμή του πίνακα που Αναφέρεται. Οι υπόλοιπες λειτουργίες αναστέλλονται μέχρι να καταλάβουν το κλείδωμα.

Κλειδώματα και πίνακες κατακερματισμού 1



συγκρούσεις

Κλειδώματα και πίνακες κατακερματισμού 2

- Η ταυτόχρονη τροποποίηση (insert, update) απαιτεί αμοιβαίο αποκλεισμό. Λύσεις:
- Αδρομερές κλείδωμα: ένα κλείδωμα για ολόκληρο τον πίνακα.
- Λεπτομερές κλείδωμα: ένα κλείδωμα ανά hash.
- Ενδιάμεση λύση: το κλείδωμα να γίνει και σε ομάδες από hashes. Σε μερικές γλώσσες δίνεται η δυνατότητα επιλογής μεγέθους ομάδας, ανάλογα με το πλήθος των hashes, τη συχνότητα των λειτουργιών και το ποσοστό συγκρούσεων.

* https://en.wikipedia.org/wiki/Concurrent_hash_table
<https://dzone.com/articles/how-concurrenthashmap-works-internally-in-java>

Κλειδώματα και στοίβες

- Χρησιμοποιούμε το ίδιο κλείδωμα για διαφορετικές μεταβλητές, όταν οι τροποποιήσεις των μεταβλητών αυτών γίνονται πάντα μαζί. Για παράδειγμα οι `stack[]`, `top` μπορεί να έχουν κοινή αντιμετώπιση:

```
int[] stack;  
int top;  
lock_type l;  
  
push(int x) {  
    Lock(l);  
    top = top + 1;  
    stack[top] = x;  
    Unlock(l);  
}
```

```
int pop() {  
    Lock(l) ;  
    int x = stack[top];  
    top = top - 1;  
    Unlock(l)  
    return x;  
}
```

Η υλοποίηση μπορεί να επεκταθεί και σε στοίβα με χρήση δεικτών.

Κλειδώματα και ουρές 1

- Η απλούστερη λύση: ένα κλείδωμα για όλη τη δομή (κυκλική ουρά)

```
int[] queue;
int back, front;
lock_type l;

enqueue(int x) {
    Lock(l);
    back = (back + 1)%size;
    queue[back] = x;
    Unlock(l);
}
```

```
int dequeue() {
    Lock(l);
    int x = queue[front];
    front = (front + 1)%size;
    Unlock(l);
    return x;
}
```

Η υλοποίηση μπορεί να επεκταθεί και σε ουρά με χρήση δεικτών.

Κλειδώματα και ουρές 2

- Αν θεωρήσουμε ότι πάντα ισχύει $back \neq front$, οι `queue[]`, `back`, `front` μπορούν να έχουν κάπως διαφορετική αντιμετώπιση (κυκλική ουρά):

```
int[] queue;
int back, front;
lock_type l_back, l_front;

enqueue(int x) {
    Lock(l_back);
    back = (back + 1)%size;
    queue[back] = x;
    Unlock(l_back);
}

int dequeue() {
    Lock(l_front);
    int x = queue[front];
    front = (front + 1)%size;
    Unlock(l_front);
    return x;
}
```

Η υλοποίηση μπορεί να επεκταθεί και σε ουρά με χρήση δεικτών.

Ειδικές περιπτώσεις

Δεν εξετάζουμε τις ακραίες περιπτώσεις, δηλαδή όταν

- η στοίβα ή η ουρά είναι άδειες και γίνεται προσπάθεια για ανάγνωση ή
- η στοίβα ή η ουρά είναι πλήρεις και γίνεται προσπάθεια για εγγραφή

Γενικά αυτές οι περιπτώσεις αντιμετωπίζονται

- με διαχείριση εξαιρέσεων ή με ενεργό αναμονή ή με συγχρονισμό σε συνθήκες

Κλειδώματα και δομές με δείκτες

Συνδεδεμένες λίστες, δένδρα, κλπ 1

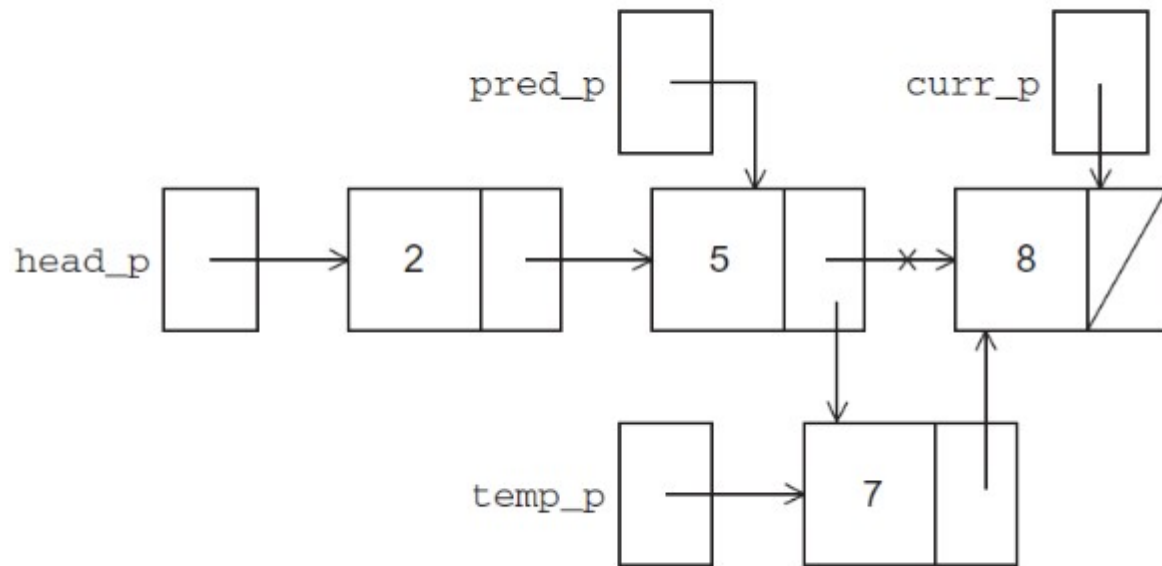
- Η ταυτόχρονη ανάγνωση είναι δυνατή αλλά η ταυτόχρονη τροποποίηση (εισαγωγή ή διαγραφή κόμβου) είναι περίπλοκη*.
- Η πιθανή ταυτόχρονη τροποποίηση γειτονικών κόμβων προκαλεί προβλήματα.

* Δες πχ https://en.wikipedia.org/wiki/Non-blocking_linked_list

- Γενικά σε δομές που χρησιμοποιούν δείκτες (πχ δένδρα, γράφοι κλπ) η ταυτόχρονη τροποποίηση (εισαγωγή ή διαγραφή κόμβου) απαιτεί προσοχή: πρέπει να διασφαλίζεται ότι δεν επηρεάζονται γειτονικοί κόμβοι.

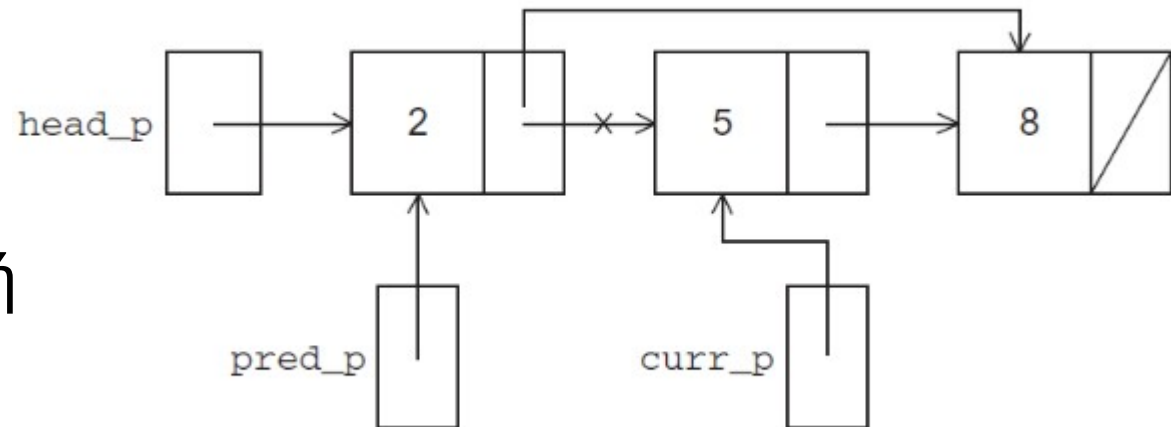
Κλειδώματα και δομές με δείκτες

Συνδεδεμένες λίστες, δένδρα κλπ 2



Εισαγωγή

Διαγραφή



Βιβλιοθήκες ταυτόχρονων δομών δεδομένων 1

- Για αρκετές από τις δομές δεδομένων που παρουσιάζονται, σε πολλές γλώσσες υπάρχουν βιβλιοθήκες ταυτόχρονων δομών δεδομένων που αποκρύπτουν τα προβλήματα υλοποίησης.
- Πχ ταυτόχρονες υλοποιήσεις για ουρές, στοίβες, πίνακες κατακερματισμού κλπ.
- Μια γλώσσα μπορεί να προσφέρει εναλλακτικές υλοποιήσεις της ίδιας δομής δεδομένων, πχ με αδρομερή ή λεπτομερή αμοιβαίο αποκλεισμό.
- Υπάρχουν και ειδικές δομές δεδομένων ειδικά για παράλληλο, ταυτόχρονο υπολογισμό.

Βιβλιοθήκες ταυτόχρονων δομών δεδομένων 2

- Οι διάφορες γλώσσες υλοποιούν ορισμένες δομές δεδομένων με διαφορετικό τρόπο. Παραδείγματα:
- ArrayLists στη Java: υλοποίηση ως array. Για τη διαχείριση ταυτοχρονισμού παρέχεται η CopyOnWriteArrayList. Κάθε τροποποίηση δημιουργεί νέο αντίγραφο της αρχικής.
- Lists στην Python: υλοποίηση ως πίνακας κατακερματισμού (hash table). Το insert είναι σχετικά thread-safe αλλά το update απαιτεί κλείδωμα λόγω πιθανής παρεμβολής.

Βιβλιοθήκες ταυτόχρονων δομών δεδομένων 3

- `/Code/ConcurrentCollections`

Κλειδώματα ανάγνωσης εγγραφής

ReadWrite Locks 1

- Όταν έχουμε δομές δεδομένων που χρησιμοποιούν αδρομερές κλείδωμα και παρουσιάζουν πολλές συχνές, σύντομες αναγνώσεις και λίγες σπάνιες “βαρύτερες” εγγραφές, τότε έχουμε το εξής φαινόμενο:
- Τα νήματα ανάγνωσης (αναγνώστες) ενώ δεν τροποποιούν τη δομή πρέπει να καταλάβουν το κλείδωμα για να διαβάσουν.
- Τα νήματα εγγραφής (συγγραφείς) σωστά πρέπει να καταλάβουν τη δομή και να αποκλείσουν τόσο τους αναγνώστες όσο και άλλους συγγραφείς.

Κλειδώματα ανάγνωσης εγγραφής

ReadWrite Locks 2

- Το ReadWrite Lock είναι ένα ειδικό ασύμμετρο κλείδωμα.
- Ένα ReadWrite Lock παράγει δύο κλειδώματα:
- Το ReadLock επιτρέπει την ταυτόχρονη προσπέλαση σε αυθαίρετο αριθμό νημάτων μόνο για ανάγνωση, αρκεί να μην έχει καταληφθεί το WriteLock
- Το WriteLock λειτουργεί σαν κλασσικό κλείδωμα και επιτρέπει την εγγραφή στη δομή, όμως πρέπει να περιμένει ώστε να είναι ελεύθερο και το ReadLock

Κλειδώματα ανάγνωσης εγγραφής

ReadWrite Locks 3

```
// shared linked_list s
// shared ReadWrite_lock_type l
```

Νήμα A

...

Lock.Read(l)

x=retrieve(s,key1)

Unlock.Read(l)

....

Νήμα B

...

Lock.Write(l)

remove(s,key3)

Unlock.Write(l)

...

Νήμα C

...

Lock.Read(l)

y=retrieve(s,key2)

Unlock.Read(l)

...

Τα νήματα A,C μπορούν να εκτελεστούν ταυτόχρονα. Το B πρέπει να περιμένει να τελειώσουν τα A, C. Αντίστροφα, αν εκτελείται το B τα A, C περιμένουν. Αν υπήρχε άλλο νήμα D για εγγραφή θα περίμενε τα A, C και B να τελειώσουν.

Java: ReadWriteLock

- Στο πακέτο `java.util.concurrent` ορίζεται η διεπαφή `ReadWriteLock` και μια σειρά κλάσεων που την υλοποιούν, για παράδειγμα `ReentrantReadWriteLock`.

```
public interface ReadWriteLock {  
    Lock readLock();           // returns the read  
    lock  
    Lock writeLock();          // returns the write  
    lock  
}
```

- Το `readLock` μπορεί να καταληφθεί ταυτόχρονα από πολλούς αναγνώστες, αρκεί να μην υπάρχουν συγγραφείς
- Το `writeLock` επιβάλλει αμοιβαίο αποκλεισμό μεταξύ των συγγραφέων, μπορεί να καταληφθεί από ένα συγγραφέα αρκεί να μην υπάρχουν αναγνώστες

Παράδειγμα: RWDictionary 1

```
class RWDictionary {  
    Map<String, Data> m =  
        new TreeMap<String, Data>();  
  
    ReentrantReadWriteLock rwl =  
        new ReentrantReadWriteLock();  
    Lock r = rwl.readLock();  
    Lock w = rwl.writeLock();  
  
    ...  
}
```

Παράδειγμα: RWDictionary 2

```
// Reader method (does not update the map)
public Data get(String key) {
    r.lock();
    try {
        return m.get(key);
    }
    finally { r.unlock(); }
}
```

```
// Writer method (changes the map)
public Data put(String key, Data value) {
    w.lock();
    try {
        m.put(key, value);
    }
    finally { w.unlock(); }
}
}
```



Ένθετα κλειδώματα και Αδιέξοδο

Ένθετα κλειδώματα (Netsed Locks) 1

```
class bankAccount {  
    int balance = ...;  
    lock_type l;  
    ...  
    transferTo(acc, amount) {  
        Lock(l);  
        if (get_balance() >= amount;  
            balance -= amount;  
        Lock(acc.l);  
        acc.balance += 100;  
        Unlock(acc.l);  
        Unlock(l)  
    }  
}
```

Ένθετο κλείδωμα
(Nested lock):

Ένα νήμα προσπαθεί
να καταλάβει ένα
κλείδωμα ενώ ήδη
κατέχει ένα άλλο
κλείδωμα.

Εδώ το νήμα κατέχει
το l και καταλαμβάνει
το acc.l

Ένθετα κλειδώματα (Netsed Locks) 2

// shared bankAccount X, Y

Νήμα A

...

X.transferTo(100, Y)

....

Νήμα B

...

Y.tansferTo(200, X);

....

Έστω δύο νήματα που εκτελούν τη μέθοδο μεταφοράς μεταξύ δύο λογαριασμών. Το νήμα A από τον λογαριασμό X στον Y, και το νήμα B από τον Y στον X.

Ένθετα κλειδώματα (Netsed Locks) 3

// shared bankAccount X, Y

Νήμα A

```
Lock(X.l);  
if (X.get_balance() >= 100)  
    X.balance -= 100;
```

```
Lock(Y.l); // block  
Y.balance += 100;  
Unlock(Y.l);  
Unlock(X.l);
```

Νήμα B

```
lock(Y.l);  
if (Y.get_balance() >= 200)  
    Y.balance -= 200;
```

```
Lock(X.l); // block  
X.balance += 200;  
Unlock(X.l);  
Unlock(Y.l);
```

Στην καλή περίπτωση, θα μπορούσε το Νήμα A ή το Νήμα B να 'προλάβουν' να καταλάβει και τα δύο κλειδώματα X.l, Y.l (ή Y.l, X.l). Όμως υπάρχει και η πιθανότητα τα δύο νήματα να καταλάβουν το Καθένα το πρώτο κλείδωμα και μετά να ανασταλούν στην κατάληψη του δεύτερου κλειδώματος.

Πιθανές λύσεις 1

- Χρήση αδρομερούς κλειδώματος: ένα νήμα κατέχει όλους τους λογαριασμούς κάθε φορά.

```
class altBankAccount {  
  
    int[] balance[1000];  
    lock_type l; // 1 lock  
  
    ...  
  
    transferTo(i, j, 100) {  
        Lock(l);  
        if (get_balance[i]() >= 100;  
            balance[i] -= 100;  
            balance[j] += 100;  
            Unlock(l)  
        }  
    }  
}
```

Πιθανές λύσεις 2

- Διαφορετική υλοποίηση:

```
transferTo (amount, acc){  
    withdraw(amount);    // Lock - Unlock (l)  
    acc.deposit(amount);  // Lock - Unlock (acc.l)  
}
```

Τα κλειδώματα είναι διαδοχικά, όχι ένθετα.

Πρόβλημα: ένα τρίτο νήμα μπορεί να δει ενδιάμεσα αποτελέσματα πριν ολοκληρωθεί η μεταφορά.

Πιθανές λύσεις 3

- Χρήση ελέγχου ή timeout στο κλείδωμα. Το νήμα μπορεί απλά να ελέγξει αν ένα κλείδωμα είναι δεσμευμένο ή να προσπαθήσει για timeout χρόνο και μετά να εγκαταλείψει και να προσπαθήσει αργότερα (σχετικά περίπλοκη υλοποίηση, συνεργατικό κλείδωμα).
- Διάταξη των καταλήψεων των κλειδωμάτων με συγκεκριμένη σειρά, πχ με αριθμό λογαριασμού. Αν πχ θεωρήσουμε ότι οι λογαριασμοί έχουν ένα μοναδικό (ακέραιο ή αλφαριθμητικό) πεδίο id, τότε αν $X.id < Y.id$ θα πρέπει το $Lock(X)$ πάντα να προηγείται του $Lock(Y)$.

Πιθανές λύσεις 3

```
class bankAccount {
```

```
    int id = ...;  
    int balance = ...;  
    lock_type l;
```

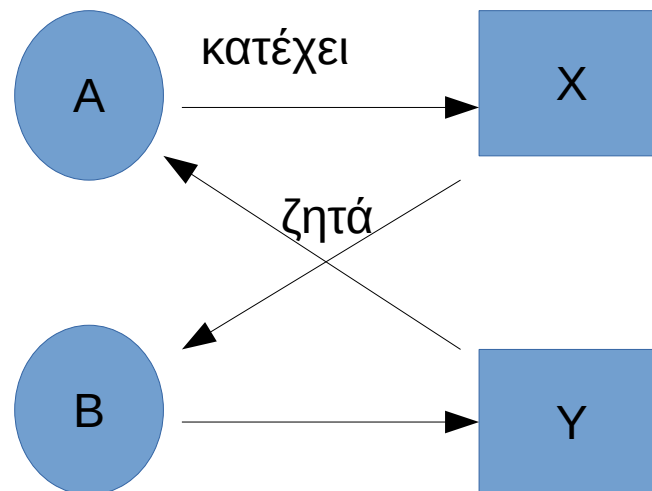
```
    ...
```

```
    transferTo(acc, amount) {  
        if (id < acc.id) {Lock(l); Lock(acc.l);}  
        else {Lock(acc.l); Lock(l);}  
        if (get_balance() >= amount;  
            balance -= amount;  
            acc.balance += 100;  
        if (this.id < acc.id) {Unock(l); Lock(acc.l);}  
        else {Unock(acc.l); Unlock(l);}  
    }
```

Στο Unlock είναι απαραίτητο να διατηρηθεί η ίδια σειρά;

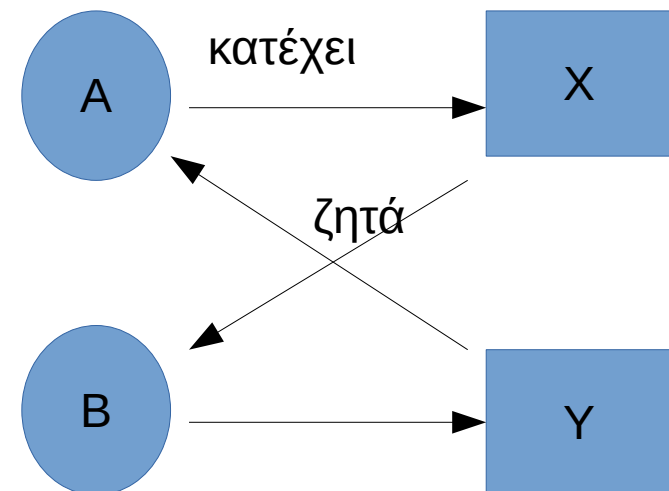
Αδιέξοδο

- Δύο νήματα A, B εκτελούν ταυτόχρονα μεταφορά ποσών:
Το A από τον λογαριασμό X στον Y.
Το B από τον λογαριασμό Y στον X.
- Το A καταλαμβάνει το κλείδωμα του X κάνει ανάληψη και ζητά το κλείδωμα του Y για να συνεχίσει.
- Το B καταλαμβάνει το κλείδωμα του Y κάνει ανάληψη και ζητά το κλείδωμα του X για να συνεχίσει.



Συνθήκες αδιεξόδου

- Τουλάχιστο δύο νήματα / διεργασίες A, B ανταγωνίζονται για τουλάχιστο δύο κρίσιμα τμήματα X, Y.
- Επιτρέπεται σταδιακή κατάληψη κρίσιμων τμημάτων με αυθαίρετη σειρά: πρώτα το X και μετά το Y ή ανάποδα.
- Δεν επιτρέπεται εγκατάλειψη κρίσιμου τμήματος λόγω χρονοκαθυστέρησης.
- Συμβαίνει κύκλος αναμονής:



Για να συμβεί αδιέξοδο
πρέπει να συναληθεύουν
και οι τέσσερις συνθήκες.

Γευματίζοντες Φιλόσοφοι

Dining Philosophers 1

- Γνωστό πρόβλημα στον ταυτόχρονο υπολογισμό που μοντελοποιεί το ζήτημα του αδιεξόδου.
- Πέντε φιλόσοφοι κάθονται γύρω από ένα τραπέζι που είναι εξοπλισμένο με πέντε πιάτα – ένα για κάθε φιλόσοφο- και πέντε πιρούνια τοποθετημένα ανά ένα μεταξύ δύο πιάτων (βλ. σχήμα).
- Ο κάθε φιλόσοφος σκέπτεται για κάποιο τυχαίο χρονικό διάστημα και μετά πρέπει να πάρει και τα δύο πιρούνια που βρίσκονται δεξιά και αριστερά (με αυτή τη σειρά) από το πιάτο του για να σερβίρει το φαγητό στο πιάτο και να γευματίσει.

Γευματίζοντες Φιλόσοφοι

Dining Philosophers 2

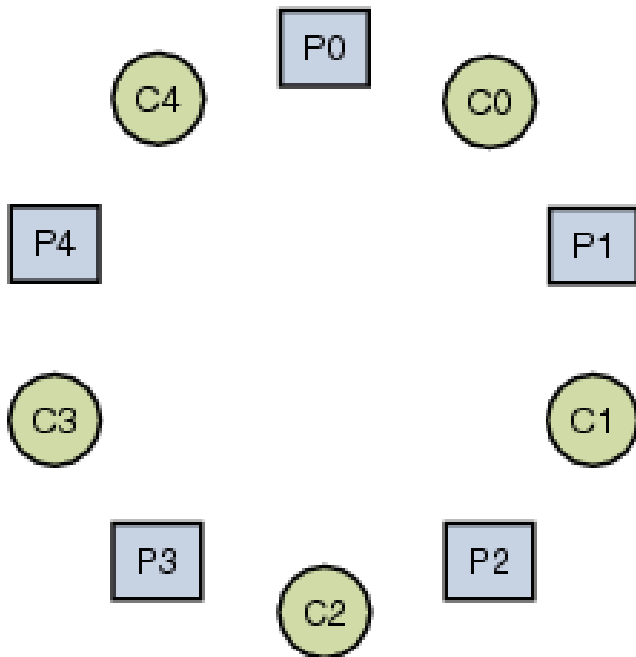
```
//shared variable  
lock_type c[5] = {1,1,1,1,1};
```

```
Philosopher(i){ //i=0,1,2...,4
```

```
while (true){  
    think();  
    Lock(c[i]);  
    Lock(c[(i+1)%5]);  
    eat();  
    Unlock(c[(i+1)%5]);  
    Unlock(c[i]);  
}
```

```
}
```

P = Philosopher
C = Chopstick



Γευματίζοντες Φιλόσοφοι

Dining Philosophers: συνθήκες αδιεξόδου

- Αμοιβαίος Αποκλεισμός: 5 δυαδικοί σηματοφόροι με αρχική τιμή 1 (ή κλειδώματα), το πιρούνι $[i]$, $i = 0, 1, \dots, 4$.
- Σταδιακή κατάληψη πόρων: Ο φιλόσοφος $[i]$, καταλαμβάνει πρώτα το πιρούνι $[i]$ και μετά το πιρούνι $[(i+1) \% 5]$.
- Μη-εγκατάληψη: Ο φιλόσοφος $[i]$ δεν αφήνει το πιρούνι $[i]$ παρά μόνο αν καταλάβει και το πιρούνι $[(i+1) \% 5]$ και καταφέρει να γευματίσει.
- Κύκλος αναμονής (wait-for cycle): Μπορεί να συμβεί το εξής- κάθε φιλόσοφος $[i]$, $i = 0, 1, \dots, 4$ έχει καταλάβει το πιρούνι $[i]$ και προσπαθεί να καταλάβει το πιρούνι $[(i+1) \% 5]$.



Γευματίζοντες Φιλόσοφοι Dining Philosophers: κώδικας

- /Code/Dining

Γευματίζοντες Φιλόσοφοι

Dining Philosophers: επίλυση

- Αποτροπή σταδιακής κατάληψης πόρων: Ένας επιπλέον μηχανισμός αμοιβαίου αποκλεισμού (σερβιτόρος!) ελέγχει την πρόσβαση των φιλοσόφων στα πιρούνια.
- Εγκατάλειψη πόρου: Ο φιλόσοφος $[i]$ ελευθερώνει το πιρούνι $[i]$ αν μετά από συγκεκριμένο χρόνο δεν καταλάβει και το πιρούνι $[(i+1) \% 5]$.
- Αποτροπή κύκλου αναμονής: Η συνηθέστερη λύση!
 - (α) Διαχωρισμός των φιλοσόφων σε άρτιους και περιττούς. Οι άρτιοι ξεκινούν από το αριστερό πιρούνι και οι περιττοί από το δεξιό.
 - (β) Αρίθμηση πόρων: όλοι οι φιλόσοφοι πρέπει να καταλαμβάνουν τα πιρούνια με αύξουσα σειρά.

Απλοί κανόνες αποφυγής αδιεξόδου

- Όταν έχουμε σταδιακή κατάληψη πόρων δηλαδή ένθετους μηχανισμούς αμοιβαίου αποκλεισμού

Ελέγχουμε για πιθανό αδιέξοδο, δηλαδή ελέγχουμε την συναλήθεια των συνθηκών αδιεξόδου.

- Αρίθμηση πόρων = αρίθμηση μοιραζόμενων μεταβλητών ή δομών δεδομένων (και των αντίστοιχων μηχανισμών αμοιβαίου αποκλεισμού):

Πάντα θα πρέπει η κατάληψη των μηχανισμών αμοιβαίου αποκλεισμού να γίνεται με ίδια (αύξουσα ή φθίνουσα) σειρά.

Πχ: αριθμοδείκτες σε πίνακες, ιεραρχία σε δένδρα, αναφορές σε λίστες..

Livelock

- Τύπος αδιεξόδου, όπου τα εμπλεκόμενα νήματα, δεν αναστέλλονται άμεσα αλλά εκτελούν κυκλικές ενέργειες, δηλαδή μετά από μερικά βήματα επανέρχονται στην αρχική τους κατάσταση.
- Στο παράδειγμα των γευματιζόντων φιλοσόφων: εφαρμόζουμε τεχνική εγκατάλειψης πόρου (με timeout) αλλά με λανθασμένο τρόπο, ώστε μετά την εγκατάλειψη τα νήματα απλά ανταλλάσσουν κατειλημμένους πόρους και πάλι δεν μπορούν να προχωρήσουν.
- Λύση: επιβολή τυχαιότητας στη διάρκεια του timeout (exponential backoff).

Starvation (Λιμοκτονία)

- Έλλειψη δικαιοσύνης στην σειρά κατάληψης πόρων.
- Αν η υλοποίηση του κλειδώματος δεν έχει πολιτική δικαιοσύνης, τότε μπορεί κάποιο νήμα, για τυχαίους λόγους να καθυστερήσει πολύ περισσότερο στην κατάληψη ενός πόρου από άλλα που ανταγωνίζεται.
- Στο παράδειγμα των γευματιζόντων φιλοσόφων, μπορεί ένας φιλόσοφος να χάνει συνεχώς την σειρά του από άλλους φιλοσόφους.
- Λύση: επιβολή δικαιοσύνης στην υλοποίηση ή στο σύστημα χρόνου εκτέλεσης (δρομολόγηση).